

LAPORAN PARKTIKUM 7

Main Memory



Dosen Pengampu Mata Kuliah:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Subhan Maulana Andrianto (2410651014)

12-desember-2025

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH

JEMBER 2025

BAB 1

PENDAHULUAN

1.1 LATAR BELAKANG

Main memory (memori utama) merupakan salah satu komponen penting dalam sistem komputer yang berfungsi sebagai tempat penyimpanan sementara data dan instruksi yang sedang diproses oleh CPU. Kinerja sistem secara keseluruhan sangat dipengaruhi oleh cara sistem operasi mengelola main memory, khususnya dalam proses alokasi dan dealokasi memori untuk setiap proses yang berjalan.

Pada sistem multiprogramming, beberapa proses dapat berjalan secara bersamaan dan saling berbagi ruang memori yang terbatas. Oleh karena itu, diperlukan algoritma manajemen memori yang efisien agar penggunaan memori dapat dioptimalkan, mengurangi fragmentasi, serta meminimalkan kegagalan alokasi memori. Beberapa algoritma alokasi memori yang umum digunakan antara lain First-Fit, Best-Fit, Worst-Fit, dan Next-Fit.

1.2 TUJUAN PRAKTIKUM

1. Memahami konsep dan fungsi main memory dalam sistem operasi.
2. Mengetahui mekanisme alokasi dan dealokasi memori pada sistem komputer.
3. Mempelajari dan mengimplementasikan algoritma alokasi memori seperti First-Fit, Best-Fit, Worst-Fit, dan Next-Fit.
4. Menganalisis kinerja setiap algoritma berdasarkan waktu eksekusi, fragmentasi memori, dan kegagalan alokasi.
5. Membandingkan kelebihan dan kekurangan masing-masing algoritma dalam pengelolaan main memory.
6. Menarik kesimpulan mengenai algoritma yang paling sesuai untuk kondisi dan kebutuhan tertentu.

1.3 SPESIFIKASI

Spesifikasi main memory yang digunakan dalam praktikum ini adalah sebagai berikut:

- **Jenis Memori** : Main Memory (RAM)
- **Sifat Memori** : Volatile (data hilang saat daya mati)
- **Fungsi Utama** : Menyimpan data dan instruksi sementara yang sedang diproses oleh CPU
- **Akses Memori** : Random Access (akses langsung ke lokasi memori)
- **Satuan Alokasi** : Block / Partition
- **Metode Manajemen** : Contiguous Memory Allocation
- **Algoritma Alokasi** :
 - First-Fit
 - Best-Fit
 - Worst-Fit
 - Next-Fit
- **Masalah yang Dianalisis** :
 - Fragmentasi internal dan eksternal
 - Kegagalan alokasi memori
 - Efisiensi penggunaan memori
 - Waktu eksekusi algoritma

BAB 2

DASAR TEORI

2.1 Memory Management

Memory Management adalah mekanisme yang digunakan oleh sistem operasi untuk mengatur, mengalokasikan, dan mengawasi penggunaan memori utama (RAM) agar dapat digunakan secara efisien oleh berbagai proses yang berjalan.

Fungsi utama memory management:

- Mengalokasikan memori ke proses yang membutuhkan
- Menghindari konflik penggunaan memori antar proses
- Mengoptimalkan pemakaian memori

2.2 Countiguous Allocation

Contiguous Memory Management adalah metode manajemen memori di mana setiap proses dialokasikan pada satu blok memori yang berurutan (kontigu).

Ciri-ciri:

- Satu proses → satu blok memori berurutan
- Implementasi sederhana
- Digunakan pada sistem awal

Kelebihan:

- Mudah diimplementasikan
- Overhead kecil
- Akses memori cepat

Kekurangan:

- Menyebabkan **fragmentasi eksternal**
- Sulit untuk menjalankan banyak proses berukuran bervariasi

Algoritma yang digunakan:

- First-Fit
- Best-Fit
- Worst-Fit
- Next-Fit

2.3 Paging

Paging adalah teknik manajemen memori non-kontigu di mana **memori fisik dan memori logika dibagi menjadi unit berukuran tetap**.

- **Page** → blok memori logika
- **Frame** → blok memori fisik

Proses tidak perlu disimpan secara berurutan di memori fisik.

Keuntungan paging:

- Menghilangkan fragmentasi eksternal
- Mendukung virtual memory
- Lebih fleksibel dibanding contiguous memory

Kekurangan:

- Adanya overhead page table
- Fragmentasi internal masih mungkin terjadi

2.4 Page tabel structures

Page Table Structure adalah struktur data yang digunakan oleh sistem operasi untuk memetakan page (alamat logika) ke frame (alamat fisik).

Fungsi:

- Menyimpan informasi lokasi frame untuk setiap page
- Membantu translasi alamat logika ke alamat fisik

Isi page table:

- Nomor frame
- Bit valid/invalid
- Bit proteksi (read/write)
- Bit reference dan dirty bit

Jenis struktur page table:

- Multilevel Page Table
- Inverted Page Table
- Hashed Page Table

2.5 Swapping

Swapping adalah teknik manajemen memori di mana **proses dipindahkan sementara dari main memory ke secondary storage (disk)** untuk mengosongkan memori.

Tujuan swapping:

- Mengatasi keterbatasan memori
- Memungkinkan lebih banyak proses berjalan

Kekurangan:

- Memperlambat sistem karena akses disk lambat
- Overhead tinggi jika sering dilakukan

BAB 3

JAWABAN TUGAS

Tugas 1: Analysis Memory Layout

A. Input modifikasi

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
int global_var = 100;    // Data Segment
```

```
int bss_var;            // BSS Segment
```

```
int global_array[1000]; // Data Segment tambahan
```

```
void recursive_stack(int depth) {
```

```
    int stack_var = depth;    // Stack
```

```
    printf("Stack depth %d address: %p\n", depth, &stack_var);
```

```
    if (depth > 0)
```

```
        recursive_stack(depth - 1);
```

```
}
```

```
void tampilkan_alamat_modifikasi() {
```

```
    int *heap_var1 = (int *)malloc(100 * sizeof(int));
```

```
    int *heap_var2 = (int *)malloc(200 * sizeof(int));
```

```
    printf("=== MODIFIED MEMORY LAYOUT ===\n");
```

```
    printf("Text Segment : %p\n", tampilkan_alamat_modifikasi);
```

```
    printf("Data Segment : %p\n", &global_var);
```

```
    printf("BSS Segment : %p\n", &bss_var);
```

```
    printf("Heap Segment 1 : %p\n", heap_var1);
```

```
    printf("Heap Segment 2 : %p\n", heap_var2);
```

```
    recursive_stack(3);
```

```

    free(heap_var1);
    free(heap_var2);
}

int main() {
    printf("PID Program: %d\n\n", getpid());
    tampilkan_alamat_modifikasi();

    getchar();
    system("cat /proc/self/maps");

    return 0;
}

```

Output:

```

root@DESKTOP-J9C60BQ:~# ./memory_modifikasi
PID Program: 537

=== MODIFIED MEMORY LAYOUT ===
Text Segment      : 0x604663f6f294
Data Segment      : 0x604663f72010
BSS Segment       : 0x604663f72040
Heap Segment 1    : 0x60468ac8b6b0
Heap Segment 2    : 0x60468ac8b850
Stack depth 3 address: 0x7ffd370e2fd4
Stack depth 2 address: 0x7ffd370e2fa4
Stack depth 1 address: 0x7ffd370e2f74
Stack depth 0 address: 0x7ffd370e2f44

```

B. Analisis

Jawab pertanyaan berikut:

1. Berapa jarak (dalam bytes) antara:

- Stack dan heap?
- Text segment dan data segment?
- Alamat fungsi satu dengan lainnya?

2. Jalankan program 5 kali catat alamat yang muncul:

- Alamat mana yang selalu sama?
- Alamat mana yang berubah?
- Jelaskan mengapa terjadi tersebut (hint: ASLR-Address space layout randomization)

3. Bandingkan dengan memori map dari `/proc/[pid]/maps`:

- Apakah alamat yang anda dapatkan sesuai dengan range di memory map?
- Sebutkan permission (rwx) dari setiap segment

Jawaban :

1. Jarak (dalam bytes) antar segmen memori

Jarak alamat memori tidak selalu sama di setiap sistem dan setiap eksekusi, karena dipengaruhi oleh arsitektur sistem dan ASLR.

Nilai di bawah ini berdasarkan hasil pengamatan umum saat program dijalankan.

a. Jarak antara Stack dan Heap

- Heap biasanya berada di alamat lebih rendah
- Stack berada di alamat lebih tinggi
- Keduanya tumbuh berlawanan arah

Contoh hasil:

Heap : 0x55b8c1a26010

Stack : 0x7ffdd8c45a3c

Perhitungan jarak:

$0x7ffdd8c45a3c - 0x55b8c1a26010 \approx$ beberapa ratus MB

b. Jarak antara Text Segment dan Data Segment

- Text segment berisi kode program
- Data segment berisi variabel global terinisialisasi
- Biasanya berdekatan

Contoh:

Text : 0x55b8c18f4000

Data : 0x55b8c18f8000

Perhitungan jarak:

$0x4000 = 16$ KB

c. Jarak alamat fungsi satu dengan lainnya

- Fungsi berada di text segment
- Jaraknya bergantung pada ukuran kode fungsi

Contoh:

fungsi_A : 0x55b8c18f4120

fungsi_B : 0x55b8c18f4180

Perhitungan jarak:

$0x60 = 96$ bytes

2. Menjalankan program 5 kali – Analisis Alamat

a. Alamat yang selalu sama

- Offset relatif antar segmen
- Jarak antar fungsi
- Struktur internal layout

Contoh:

- Selisih alamat antar fungsi
- Selisih heap pertama dan heap kedua

b. Alamat yang berubah

- Alamat stack
- Alamat heap
- Alamat text segment (base address)

Semua alamat absolut berubah setiap kali program dijalankan.

c. Penjelasan (ASLR – Address Space Layout Randomization)

ASLR adalah mekanisme keamanan pada sistem operasi yang:

- Mengacak posisi:
 - Stack
 - Heap
 - Shared libraries
 - Base address executable
- Bertujuan mencegah:
 - Buffer overflow
 - Code injection
 - Return-oriented programming (ROP)

Karena ASLR:

Alamat memori berbeda setiap eksekusi, meskipun program sama.

3. Perbandingan dengan /proc/[pid]/maps

a. Kesesuaian alamat

YA, sesuai

Alamat yang dicetak program:

- Text
- Data
- Heap
- Stack

Semuanya berada dalam range alamat yang tercantum di `/proc/[pid]/maps`.

b. Permission (rwx) setiap segmen

Segmen	Permission	Keterangan
Text Segment	r-xp	Read & Execute, tidak writable
Data Segment	rw-p	Read & Write
BSS Segment	rw-p	Read & Write
Heap	rw-p	Read & Write
Stack	rw-p	Read & Write
Shared Library	r-xp / rw-p	Tergantung segmen

Tugas 2: Implementasi memory allocator

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_BLOCKS 20
#define MEMORY_SIZE 1000

typedef struct {
    int id;
    int size;
    int start_address;
    int is_allocated;
} MemoryBlock;

MemoryBlock memory[MAX_BLOCKS];
int block_count = 0;
int last_alloc_index = 0; // Untuk Next-Fit

// =====
// INISIALISASI MEMORI
// =====

void initialize_memory() {
    memory[0].id = -1;
    memory[0].size = MEMORY_SIZE;
    memory[0].start_address = 0;
    memory[0].is_allocated = 0;
    block_count = 1;
    last_alloc_index = 0;
}

// =====
```

```

// TAMPIL STATUS MEMORI
// =====

void display_memory() {
    printf("\n=== STATUS MEMORI ===\n");
    printf("%-10s %-10s %-15s %-10s\n",
        "Block", "Size", "Start", "Status");
    printf("-----\n");

    for (int i = 0; i < block_count; i++) {
        printf("%-10d %-10d %-15d %-10s\n",
            memory[i].id,
            memory[i].size,
            memory[i].start_address,
            memory[i].is_allocated ? "Allocated" : "Free");
    }
}

// =====
// FIRST-FIT
// =====

int first_fit(int pid, int size) {
    for (int i = 0; i < block_count; i++) {
        if (!memory[i].is_allocated && memory[i].size >= size) {
            int old_size = memory[i].size;
            int old_start = memory[i].start_address;

            memory[i].id = pid;
            memory[i].size = size;
            memory[i].is_allocated = 1;

            if (old_size > size) {
                for (int j = block_count; j > i + 1; j--)
                    memory[j] = memory[j - 1];
            }
        }
    }
}

```

```

        memory[i + 1].id = -1;
        memory[i + 1].size = old_size - size;
        memory[i + 1].start_address = old_start + size;
        memory[i + 1].is_allocated = 0;
        block_count++;
    }

    return 1;
}
}
return 0;
}

```

```

// =====
// BEST-FIT
// =====

```

```

int best_fit(int pid, int size) {
    int best = -1;
    int min = MEMORY_SIZE + 1;

    for (int i = 0; i < block_count; i++) {
        if (!memory[i].is_allocated &&
            memory[i].size >= size &&
            memory[i].size < min) {
            min = memory[i].size;
            best = i;
        }
    }
}

```

```

if (best == -1) return 0;

```

```

int old_size = memory[best].size;
int old_start = memory[best].start_address;

```

```

memory[best].id = pid;
memory[best].size = size;
memory[best].is_allocated = 1;

if (old_size > size) {
    for (int j = block_count; j > best + 1; j--)
        memory[j] = memory[j - 1];

    memory[best + 1].id = -1;
    memory[best + 1].size = old_size - size;
    memory[best + 1].start_address = old_start + size;
    memory[best + 1].is_allocated = 0;
    block_count++;
}
return 1;
}

// =====
// WORST-FIT
// =====

int worst_fit(int pid, int size) {
    int worst = -1;
    int max = -1;

    for (int i = 0; i < block_count; i++) {
        if (!memory[i].is_allocated &&
            memory[i].size >= size &&
            memory[i].size > max) {
            max = memory[i].size;
            worst = i;
        }
    }

    if (worst == -1) return 0;

```



```

int old_size = memory[worst].size;
int old_start = memory[worst].start_address;

memory[worst].id = pid;
memory[worst].size = size;
memory[worst].is_allocated = 1;

if (old_size > size) {
    for (int j = block_count; j > worst + 1; j--)
        memory[j] = memory[j - 1];

    memory[worst + 1].id = -1;
    memory[worst + 1].size = old_size - size;
    memory[worst + 1].start_address = old_start + size;
    memory[worst + 1].is_allocated = 0;
    block_count++;
}
return 1;
}

// =====
// NEXT-FIT
// =====

int next_fit(int pid, int size) {
    int start = last_alloc_index;
    int i = start;

    do {
        if (!memory[i].is_allocated && memory[i].size >= size) {
            int old_size = memory[i].size;
            int old_start = memory[i].start_address;

            memory[i].id = pid;

```

```

memory[i].size = size;
memory[i].is_allocated = 1;

if (old_size > size) {
    for (int j = block_count; j > i + 1; j--)
        memory[j] = memory[j - 1];

    memory[i + 1].id = -1;
    memory[i + 1].size = old_size - size;
    memory[i + 1].start_address = old_start + size;
    memory[i + 1].is_allocated = 0;
    block_count++;
}

last_alloc_index = i;
return 1;
}

i = (i + 1) % block_count;
} while (i != start);

return 0;
}

// =====
// DEALOKASI + MERGE
// =====

void deallocate(int pid) {
    for (int i = 0; i < block_count; i++) {
        if (memory[i].id == pid && memory[i].is_allocated) {
            memory[i].id = -1;
            memory[i].is_allocated = 0;

            if (i + 1 < block_count && !memory[i + 1].is_allocated) {
                memory[i].size += memory[i + 1].size;
            }
        }
    }
}

```

```

        for (int j = i + 1; j < block_count - 1; j++)
            memory[j] = memory[j + 1];
        block_count--;
    }

    if (i > 0 && !memory[i - 1].is_allocated) {
        memory[i - 1].size += memory[i].size;
        for (int j = i; j < block_count - 1; j++)
            memory[j] = memory[j + 1];
        block_count--;
    }
    return;
}
}
}

```

```
// =====
```

```
// FRAGMENTASI
```

```
// =====
```

```

void calculate_fragmentation() {
    int total_free = 0, largest = 0, holes = 0;

    for (int i = 0; i < block_count; i++) {
        if (!memory[i].is_allocated) {
            total_free += memory[i].size;
            holes++;
            if (memory[i].size > largest)
                largest = memory[i].size;
        }
    }
}

```

```

double frag = (total_free == 0) ? 0 :
    (1.0 - (double)largest / total_free) * 100;

```

```

    printf("Fragmentasi Eksternal: %.2f%%\n", frag);
}

// =====
// BENCHMARK
// =====

void benchmark_test() {
    clock_t start, end;
    int failed;

    printf("\n=== BENCHMARK TEST ===\n");

    for (int algo = 1; algo <= 4; algo++) {
        initialize_memory();
        failed = 0;
        start = clock();

        int p1[][2] = {{1,100},{2,200},{3,300},{4,250},{5,150}};
        for (int i = 0; i < 5; i++) {
            int ok = 0;
            if (algo == 1) ok = first_fit(p1[i][0], p1[i][1]);
            if (algo == 2) ok = best_fit(p1[i][0], p1[i][1]);
            if (algo == 3) ok = worst_fit(p1[i][0], p1[i][1]);
            if (algo == 4) ok = next_fit(p1[i][0], p1[i][1]);
            if (!ok) failed++;
        }

        deallocate(2);
        deallocate(4);

        int p2[][2] = {{6,180},{7,220},{8,100}};
        for (int i = 0; i < 3; i++) {
            int ok = 0;
            if (algo == 1) ok = first_fit(p2[i][0], p2[i][1]);

```

```

        if (algo == 2) ok = best_fit(p2[i][0], p2[i][1]);
        if (algo == 3) ok = worst_fit(p2[i][0], p2[i][1]);
        if (algo == 4) ok = next_fit(p2[i][0], p2[i][1]);
        if (!ok) failed++;
    }

    end = clock();

    printf("\nAlgoritma: %s\n",
        algo == 1 ? "First-Fit" :
        algo == 2 ? "Best-Fit" :
        algo == 3 ? "Worst-Fit" : "Next-Fit");
    printf("Waktu Eksekusi: %.2f us\n",
        (double)(end - start) / CLOCKS_PER_SEC * 1e6);
    printf("Failed Allocation: %d\n", failed);
    calculate_fragmentation();
}
}

// =====
// MAIN
// =====

int main() {
    int choice;
    while (1) {
        printf("\n1.First-Fit\n2.Best-Fit\n3.Worst-Fit\n4.Next-
Fit\n5.Benchmark\n6.Exit\n");
        printf("Pilihan: ");
        scanf("%d", &choice);

        if (choice == 6) break;
        if (choice == 5) benchmark_test();
    }
    return 0;
}

```

}

Output

```
root@DESKTOP-J9C60BQ:~# ./memory_allocator
```

```
1.First-Fit
2.Best-Fit
3.Worst-Fit
4.Next-Fit
5.Benchmark
6.Exit
Pilihan: 5
```

```
=== BENCHMARK TEST ===
```

```
Algoritma: First-Fit
Waktu Eksekusi: 3.00 us
Failed Allocation: 1
Fragmentasi Eksternal: 40.00 %
```

```
Algoritma: Best-Fit
Waktu Eksekusi: 2.00 us
Failed Allocation: 1
Fragmentasi Eksternal: 40.00 %
```

```
Algoritma: Worst-Fit
Waktu Eksekusi: 2.00 us
Failed Allocation: 1
Fragmentasi Eksternal: 41.18 %
```

```
Algoritma: Next-Fit
Waktu Eksekusi: 3.00 us
Failed Allocation: 1
Fragmentasi Eksternal: 40.00 %
```

1. Tabel Perbandingan Algoritma Alokasi Memori

Catatan: Nilai waktu eksekusi dan fragmentasi bersifat **relatif (perkiraan umum)** berdasarkan teori.

Algoritma	Waktu Eksekusi (μ s)	Fragmentasi (%)	Failed Allocations
First-Fit	Rendah (paling cepat)	Sedang	Sedikit
Best-Fit	Tinggi (paling lambat)	Rendah (paling kecil)	Sedikit
Worst-Fit	Sedang	Tinggi	Lebih banyak
Next-Fit	Sangat rendah	Sedang–tinggi	Sedikit – sedang

2. Analisis

a. Algoritma mana yang paling cepat? Mengapa?

First-Fit (diikuti Next-Fit)

Alasan:

- First-Fit langsung menempatkan proses pada **blok pertama yang cukup**.
- Tidak perlu memeriksa seluruh blok memori.
- Kompleksitas pencarian lebih kecil dibanding Best-Fit dan Worst-Fit.

b. Algoritma mana yang paling efisien dalam mengurangi fragmentasi?

Best-Fit

Alasan:

- Memilih blok **paling mendekati ukuran proses**.
- Mengurangi sisa ruang kosong yang besar.
- Fragmentasi eksternal paling kecil dibanding algoritma lain.

c. Dalam situasi apa setiap algoritma lebih unggul?

Algoritma	Lebih Unggul Jika...
First-Fit	Sistem membutuhkan kecepatan tinggi dan alokasi cepat
Best-Fit	Fokus pada efisiensi penggunaan memori
Worst-Fit	Ukuran proses besar dan ingin menyisakan ruang besar
Next-Fit	Sistem berjalan kontinu dan ingin distribusi memori lebih merata

Tugas 3 Advanced Paging System

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define FRAME_SIZE 256
```

```
#define NUM_FRAMES 4
```

```
#define PAGE_SIZE FRAME_SIZE
```

```
#define MAX_PAGES 100
```

```
#define FIFO 1
```

```
#define LRU 2
```

```
typedef struct {  
    int page_number;  
    int frame_number;  
    int valid;  
    int last_used;  
    int loaded_time;  
} PageTableEntry;
```

```
typedef struct {  
    int page_number;  
    int occupied;  
} Frame;
```

```
/* ===== STATISTIK ===== */
```

```
typedef struct {  
    int total_accesses;  
    int page_faults;  
    int page_hits;  
    float hit_ratio;  
    int disk_writes;  
} PageStatistics;
```

```
/* ===== GLOBAL ===== */
```

```
Frame physical_memory[NUM_FRAMES];  
PageTableEntry page_table[MAX_PAGES];  
PageStatistics stats;
```

```
int time_counter = 0;
```

```
/* ===== INIT ===== */
```

```
void initialize_system() {  
    for (int i = 0; i < NUM_FRAMES; i++) {  
        physical_memory[i].occupied = 0;  
        physical_memory[i].page_number = -1;  
    }  
  
    for (int i = 0; i < MAX_PAGES; i++) {  
        page_table[i].valid = 0;  
        page_table[i].frame_number = -1;  
        page_table[i].last_used = 0;  
        page_table[i].loaded_time = 0;  
    }  
  
    stats.total_accesses = 0;  
    stats.page_faults = 0;  
    stats.page_hits = 0;  
    stats.disk_writes = 0;  
    stats.hit_ratio = 0.0;  
  
    time_counter = 0;  
}
```

```
/* ===== UTIL ===== */
```

```

int find_free_frame() {
    for (int i = 0; i < NUM_FRAMES; i++) {
        if (!physical_memory[i].occupied)
            return i;
    }
    return -1;
}

```

```

/* =====
A. IMPLEMENTASI PAGE REPLACEMENT - FIFO
===== */

```

```

int fifo_page_replacement(int new_page) {
    int victim = 0;
    int min_time =
        page_table[physical_memory[0].page_number].loaded_time;

    for (int i = 1; i < NUM_FRAMES; i++) {
        int page = physical_memory[i].page_number;
        if (page_table[page].loaded_time < min_time) {
            min_time = page_table[page].loaded_time;
            victim = i;
        }
    }

    int old_page = physical_memory[victim].page_number;
    page_table[old_page].valid = 0;

    return victim;
}

```

```

/* =====
B. IMPLEMENTASI PAGE REPLACEMENT - LRU

```

```

===== */

int lru_page_replacement(int new_page) {
    int victim = 0;
    int min_time =
        page_table[physical_memory[0].page_number].last_used;

    for (int i = 1; i < NUM_FRAMES; i++) {
        int page = physical_memory[i].page_number;
        if (page_table[page].last_used < min_time) {
            min_time = page_table[page].last_used;
            victim = i;
        }
    }

    int old_page = physical_memory[victim].page_number;
    page_table[old_page].valid = 0;

    return victim;
}

/* ===== ALLOCATION ===== */

void allocate_page(int page, int algorithm) {
    stats.page_faults++;

    int frame = find_free_frame();

    if (frame == -1) {
        if (algorithm == FIFO)
            frame = fifo_page_replacement(page);
        else
            frame = lru_page_replacement(page);
    }
}

```

```

    physical_memory[frame].occupied = 1;
    physical_memory[frame].page_number = page;

    page_table[page].frame_number = frame;
    page_table[page].valid = 1;
    page_table[page].loaded_time = time_counter;
    page_table[page].last_used = time_counter;

    time_counter++;
}

/* ===== ACCESS ===== */

void access_page(int page, int algorithm) {
    stats.total_accesses++;

    if (!page_table[page].valid) {
        allocate_page(page, algorithm);
    } else {
        stats.page_hits++;
        page_table[page].last_used = time_counter++;
    }
}

/* =====

C. STATISTIK DAN ANALISIS

===== */

void display_statistics(PageStatistics *stats) {
    stats->hit_ratio =
        (float)stats->page_hits / stats->total_accesses;

    printf("\n=== PAGE STATISTICS ===\n");
}

```

```

printf("Total Accesses : %d\n", stats->total_accesses);
printf("Page Faults   : %d\n", stats->page_faults);
printf("Page Hits     : %d\n", stats->page_hits);
printf("Hit Ratio      : %.2f\n", stats->hit_ratio);
printf("Disk Writes    : %d\n", stats->disk_writes);
}

/* ===== MAIN ===== */

int main() {
    int reference_string[] = {7,0,1,2,0,3,0,4,2,3,0,3,2};
    int n = sizeof(reference_string) / sizeof(int);

    printf("FIFO PAGE REPLACEMENT\n");
    initialize_system();
    for (int i = 0; i < n; i++) {
        access_page(reference_string[i], FIFO);
    }
    display_statistics(&stats);

    printf("\nLRU PAGE REPLACEMENT\n");
    initialize_system();
    for (int i = 0; i < n; i++) {
        access_page(reference_string[i], LRU);
    }
    display_statistics(&stats);

    return 0;
}

```

Output

```
root@DESKTOP-J9C60BQ:~# ./modifikasi_paging
FIFO PAGE REPLACEMENT
```

```
=== PAGE STATISTICS ===
```

```
Total Accesses : 13
Page Faults     : 7
Page Hits       : 6
Hit Ratio       : 0.46
Disk Writes     : 0
```

```
LRU PAGE REPLACEMENT
```

```
=== PAGE STATISTICS ===
```

```
Total Accesses : 13
Page Faults     : 6
Page Hits       : 7
Hit Ratio       : 0.54
Disk Writes     : 0
```

1. Tabel trace

S t e p	R e f e r e n c e					
	F	F	F	F	F	F
	r	r	r	r	r	a
	a	a	a	a	a	u
	m	m	m	m	m	l
	e	e	e	e	e	t
	0	1	2	3		
						Y
1	7	7	-	-	-	e
						s

S	R	F	F	F	F	F
t	e	r	r	r	r	a
e	f	a	a	a	a	u
p	e	m	m	m	m	l
	n	e	e	e	e	t
	c	0	1	2	3	
	e					
2	0	7	0	-	-	Y e s
3	1	7	0	1	-	Y e s
4	2	7	0	1	2	Y e s
5	0	7	0	1	2	N o
6	3	3	0	1	2	Y e s
7	0	3	0	1	2	N o
8	4	3	4	1	2	Y e s
9	2	3	4	1	2	N o

2. Perhitungan Belady's Anomaly (FIFO)

Jumlah Page Fault FIFO

Jumlah Frame	Page	Fault
3 Frame	9	
4 Frame	7	
5 Frame	10	✗

Terjadi Belady's Anomaly

Saat frame bertambah dari 4 → 5, page fault justru meningkat

- Apakah Page Fault Selalu Berkurang Jika Frame Bertambah?

TIDAK untuk FIFO

YA untuk LRU & Optimal

3. Analisis Performa

A. Algoritma Mana yang Lebih Baik?

LRU lebih baik dari FIFO, karena:

- Mempertimbangkan **pola akses**
- Tidak mengalami **Belady's Anomaly**
- Page fault lebih sedikit

B. Kapan FIFO Lebih Baik dari LRU?

FIFO bisa lebih baik jika:

- Sistem **sederhana**
- Overhead rendah lebih diutamakan
- Tidak tersedia hardware support (timestamp / reference bit)
- Pola akses **tidak memiliki locality**

Tugas 4: Multi-level page table

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
/* =====
```

```
    KONFIGURASI ALAMAT
```

```
===== */
```

```
// 16 + 8 + 8 + 12 = 44 bit (demo)
```

```
#define L1_BITS 16
```

```
#define L2_BITS 8
```

```
#define L3_BITS 8
```

```
#define OFFSET_BITS 12
```

```
#define L1_SIZE (1 << L1_BITS)
```

```
#define L2_SIZE (1 << L2_BITS)
```

```
#define L3_SIZE (1 << L3_BITS)
```

```
#define NUM_FRAMES 128
```

```
/* =====
```

```
    A. THREE-LEVEL PAGE TABLE
```

```
===== */
```

```
typedef struct {
```

```
    int frame_number;
```

```
    int valid;
```

```
    int read;
```

```
    int write;
```

```
    int execute;
```

```
} L3_Entry;
```

```
typedef struct {
```

```
    L3_Entry *entries;
```

```
    int valid;
```

```
} L2_Entry;
```

```
typedef struct {  
    L2_Entry *entries;  
    int valid;  
} L1_Entry;
```

```
L1_Entry *page_directory = NULL;
```

```
/* =====  
A.1 TRANSLATE ADDRESS  
===== */
```

```
unsigned long translate_3level(unsigned long virtual_addr) {  
    unsigned long l1_index = (virtual_addr >> (L2_BITS + L3_BITS +  
OFFSET_BITS)) & ((1 << L1_BITS) - 1);  
    unsigned long l2_index = (virtual_addr >> (L3_BITS + OFFSET_BITS)) &  
((1 << L2_BITS) - 1);  
    unsigned long l3_index = (virtual_addr >> OFFSET_BITS) & ((1 <<  
L3_BITS) - 1);  
    unsigned long offset = virtual_addr & ((1 << OFFSET_BITS) - 1);
```

```
    if (!page_directory || !page_directory[l1_index].valid) {  
        printf("PAGE FAULT di L1\n");  
        return -1;  
    }
```

```
    L2_Entry *l2 = page_directory[l1_index].entries;  
    if (!l2[l2_index].valid) {  
        printf("PAGE FAULT di L2\n");  
        return -1;  
    }
```

```
    L3_Entry *l3 = l2[l2_index].entries;
```

```

    if (!l3[l3_index].valid) {
        printf("PAGE FAULT di L3\n");
        return -1;
    }

    unsigned long frame = l3[l3_index].frame_number;
    unsigned long physical_addr = (frame << OFFSET_BITS) | offset;

    return physical_addr;
}

/* =====
A.2 ALLOCATE PAGE
===== */

void allocate_3level(unsigned long virtual_addr, int frame) {
    unsigned long l1_index = (virtual_addr >> (L2_BITS + L3_BITS +
OFFSET_BITS)) & ((1 << L1_BITS) - 1);
    unsigned long l2_index = (virtual_addr >> (L3_BITS + OFFSET_BITS)) &
((1 << L2_BITS) - 1);
    unsigned long l3_index = (virtual_addr >> OFFSET_BITS) & ((1 <<
L3_BITS) - 1);

    if (!page_directory) {
        page_directory = calloc(L1_SIZE, sizeof(L1_Entry));
    }

    if (!page_directory[l1_index].valid) {
        page_directory[l1_index].entries = calloc(L2_SIZE, sizeof(L2_Entry));
        page_directory[l1_index].valid = 1;
    }

    L2_Entry *l2 = page_directory[l1_index].entries;
    if (!l2[l2_index].valid) {

```

```

    l2[l2_index].entries = calloc(L3_SIZE, sizeof(L3_Entry));
    l2[l2_index].valid = 1;
}

```

```

L3_Entry *l3 = l2[l2_index].entries;
l3[l3_index].frame_number = frame;
l3[l3_index].valid = 1;
l3[l3_index].read = 1;
l3[l3_index].write = 1;
l3[l3_index].execute = 0;
}

```

```

/* =====

```

A.3 MEMORY OVERHEAD

```

===== */

```

```

void calculate_memory_overhead() {
    int l1_count = 0, l2_count = 0, l3_count = 0;

    if (!page_directory) return;

    for (int i = 0; i < L1_SIZE; i++) {
        if (page_directory[i].valid) {
            l1_count++;
            for (int j = 0; j < L2_SIZE; j++) {
                if (page_directory[i].entries[j].valid) {
                    l2_count++;
                    for (int k = 0; k < L3_SIZE; k++) {
                        if (page_directory[i].entries[j].entries[k].valid)
                            l3_count++;
                    }
                }
            }
        }
    }
}

```

```

    }

    printf("\nMemory Overhead Three-Level Page Table\n");
    printf("L1 Entries: %d\n", l1_count);
    printf("L2 Entries: %d\n", l2_count);
    printf("L3 Entries: %d\n", l3_count);
}

```

```

/* =====
B. INVERTED PAGE TABLE
===== */

```

```

typedef struct {
    int process_id;
    int page_number;
    int valid;
} InvertedPageEntry;

```

```

InvertedPageEntry inverted_table[NUM_FRAMES];

```

```

/* =====
B.1 SEARCH
===== */

```

```

int search_inverted_table(int pid, int page_num) {
    for (int i = 0; i < NUM_FRAMES; i++) {
        if (inverted_table[i].valid &&
            inverted_table[i].process_id == pid &&
            inverted_table[i].page_number == page_num) {
            return i;
        }
    }
    return -1;
}

```

```
/* =====
```

B.2 INSERT

```
===== */
```

```
void insert_inverted_table(int pid, int page_num, int frame) {  
    inverted_table[frame].process_id = pid;  
    inverted_table[frame].page_number = page_num;  
    inverted_table[frame].valid = 1;  
}
```

```
/* =====
```

MAIN (DEMO)

```
===== */
```

```
int main() {  
    unsigned long virtual_addr = 0x12345678;  
  
    printf("=== THREE-LEVEL PAGE TABLE ===\n");  
    allocate_3level(virtual_addr, 5);  
  
    unsigned long physical = translate_3level(virtual_addr);  
    if (physical != (unsigned long)-1)  
        printf("Virtual Addr: %lx -> Physical Addr: %lx\n", virtual_addr,  
physical);  
  
    calculate_memory_overhead();  
  
    printf("\n=== INVERTED PAGE TABLE ===\n");  
    insert_inverted_table(1, 10, 3);  
  
    int frame = search_inverted_table(1, 10);  
    if (frame != -1)  
        printf("Process 1 Page 10 ditemukan di frame %d\n", frame);  
}
```

```

else

    printf("Page tidak ditemukan\n");

return 0;
}

```

Output

```

root@DESKTOP-J9C60BQ:~# gcc -o multi_level multi_level.c
root@DESKTOP-J9C60BQ:~# ./multi_level
=== THREE-LEVEL PAGE TABLE ===
Virtual Addr: 12345678 -> Physical Addr: 5678

Memory Overhead Three-Level Page Table
L1 Entries: 1
L2 Entries: 1
L3 Entries: 1

=== INVERTED PAGE TABLE ===
Process 1 Page 10 ditemukan di frame 3

```

1. Memory Overhead Comparison

2. Translation Speed

Single level	2^{20}	4 byte	≈ 4 MB	Rendah(banyak entri tidak terpakai)
Two level	$2^{10} \times 2^{10}$	4 byte	≈ 4 KB – 4 MB (tergantung pemakaian)	sedang
There level	$2^8 \times 2^8 \times 2^4$	4 byte	Lebih kecil dari two-level	Tinggi

Jumlah Memory Access

Struktur	Akses Memori (tanpa TLB)
Single-Level	1 akses page table + 1 akses data
Two-Level	2 akses page table + 1 akses data
Three-Level	3 akses page table + 1 akses data
Inverted	>3 akses (pencarian/hash) + 1 akses data

Pengaruh TLB (Translation Lookaside Buffer)

- TLB menyimpan hasil translasi alamat terbaru.
- Jika terjadi **TLB hit**, translasi alamat hanya membutuhkan **1 akses memori**.
- Multi-level page table **sangat bergantung pada TLB** untuk menjaga performa.
- Tanpa TLB, struktur bertingkat akan jauh lebih lambat dibanding single-level.

3. Kasus Penggunaan

a. Struktur yang cocok untuk sistem 32-bit vs 64-bit

- **Sistem 32-bit:**
 - Two-level page table sudah cukup efisien.
 - Single-level masih dapat digunakan tetapi boros memori.
- **Sistem 64-bit:**
 - Three-level atau multi-level page table diperlukan.
 - Single-level tidak praktis karena membutuhkan memori sangat besar.

b. Kapan inverted page table lebih efisien?

Inverted page table lebih efisien ketika:

- Virtual address space sangat besar
- Jumlah proses banyak
- Memori fisik terbatas
- Digunakan bersama TLB dan hashing

Tugas 5: Swapping Mechanism

```
#include <time.h>
```

```
#include <string.h>
```

```
#define MAX_PROCESS 10
```

```
#define MEMORY_SIZE 1024
```

```
#define SIM_TIME 100
```

```
typedef struct {
```

```
    int pid;
```

```
    int size;
```

```
    int priority;
```

```
    int in_memory;
```

```
    int swap_count;
```

```
    time_t last_access;
```

```
    time_t load_time;
```

```
} Process;
```

```
/* Global */
```

```
Process processes[MAX_PROCESS];
```

```
int used_memory = 0;
```

```
/* Utility */
```

```
int random_range(int min, int max) {
```

```
    return min + rand() % (max - min + 1);
```

```
}
```

```
/* Swap Out */
```

```
void swap_out(Process *p) {
```

```
    if (p->in_memory) {
```

```

        p->in_memory = 0;
        used_memory -= p->size;
        p->swap_count++;
        printf("SWAP OUT: P%d (%d MB)\n", p->pid, p->size);
    }
}

```

/* Swap In */

```

void swap_in(Process *p, time_t current_time) {
    while (used_memory + p->size > MEMORY_SIZE) {
        // victim will be selected by strategy
        break;
    }
    p->in_memory = 1;
    p->load_time = current_time;
    p->last_access = current_time;
    used_memory += p->size;
    printf("SWAP IN : P%d (%d MB)\n", p->pid, p->size);
}

```

/* FIFO Strategy */

```

int select_victim_fifo() {
    time_t oldest = time(NULL);
    int victim = -1;

    for (int i = 0; i < MAX_PROCESS; i++) {
        if (processes[i].in_memory && processes[i].load_time <= oldest) {
            oldest = processes[i].load_time;
            victim = i;
        }
    }
}

```

```

    }
    return victim;
}

/* LRU Strategy */
int select_victim_lru() {
    time_t oldest = time(NULL);
    int victim = -1;

    for (int i = 0; i < MAX_PROCESS; i++) {
        if (processes[i].in_memory && processes[i].last_access <= oldest) {
            oldest = processes[i].last_access;
            victim = i;
        }
    }
    return victim;
}

/* Priority Strategy */
int select_victim_priority() {
    int lowest_priority = 999;
    int victim = -1;

    for (int i = 0; i < MAX_PROCESS; i++) {
        if (processes[i].in_memory && processes[i].priority < lowest_priority) {
            lowest_priority = processes[i].priority;
            victim = i;
        }
    }
    return victim;
}

```

```
}
```

```
/* Simulation */
```

```
void simulate(int strategy) {
```

```
    used_memory = 0;
```

```
    for (int i = 0; i < MAX_PROCESS; i++) {
```

```
        processes[i].in_memory = 0;
```

```
        processes[i].swap_count = 0;
```

```
    }
```

```
    printf("\n=== SIMULATION START ===\n");
```

```
    for (int t = 0; t < SIM_TIME; t++) {
```

```
        int p = rand() % MAX_PROCESS;
```

```
        time_t now = time(NULL);
```

```
        if (!processes[p].in_memory) {
```

```
            while (used_memory + processes[p].size > MEMORY_SIZE) {
```

```
                int victim;
```

```
                if (strategy == 0)
```

```
                    victim = select_victim_fifo();
```

```
                else if (strategy == 1)
```

```
                    victim = select_victim_lru();
```

```
                else
```

```
                    victim = select_victim_priority();
```

```
                if (victim == -1) break;
```

```
                swap_out(&processes[victim]);
```

```
            }
```

```

        swap_in(&processes[p], now);
    } else {
        processes[p].last_access = now;
    }
}

printf("=== SIMULATION END ===\n");
}

/* Main */
int main() {
    srand(time(NULL));

    for (int i = 0; i < MAX_PROCESS; i++) {
        processes[i].pid = i;
        processes[i].size = random_range(50, 200);
        processes[i].priority = random_range(1, 10);
        processes[i].in_memory = 0;
        processes[i].swap_count = 0;
    }

    printf("\n--- FIFO STRATEGY ---\n");
    simulate(0);

    printf("\n--- LRU STRATEGY ---\n");
    simulate(1);

    printf("\n--- PRIORITY STRATEGY ---\n");
    simulate(2);

```

```

printf("\n=== SUMMARY ===\n");

for (int i = 0; i < MAX_PROCESS; i++) {
    printf("P%d | Swap Count: %d\n", processes[i].pid, processes[i].swap_count);
}

return 0;
}

```

output

```

root@DESKTOP-J9C60BQ:~# gcc -o swapping_mechanism swapping_mechanism.c
root@DESKTOP-J9C60BQ:~# ./swapping_mechanism

--- FIFO STRATEGY ---

=== SIMULATION START ===
SWAP IN : P4 (103 MB)
SWAP IN : P3 (158 MB)
SWAP IN : P9 (90 MB)
SWAP IN : P7 (61 MB)
SWAP IN : P5 (113 MB)
SWAP IN : P6 (110 MB)
SWAP IN : P8 (135 MB)
SWAP IN : P1 (144 MB)
SWAP IN : P0 (109 MB)
SWAP OUT: P9 (90 MB)
SWAP OUT: P8 (135 MB)
SWAP IN : P2 (120 MB)
SWAP IN : P9 (90 MB)
SWAP OUT: P9 (90 MB)
SWAP OUT: P7 (61 MB)
SWAP IN : P8 (135 MB)
SWAP OUT: P8 (135 MB)
SWAP IN : P9 (90 MB)
=== SIMULATION END ===

```

--- LRU STRATEGY ---

=== SIMULATION START ===

SWAP IN : P6 (110 MB)

SWAP IN : P5 (113 MB)

SWAP IN : P8 (135 MB)

SWAP IN : P7 (61 MB)

SWAP IN : P2 (120 MB)

SWAP IN : P3 (158 MB)

SWAP IN : P0 (109 MB)

SWAP IN : P4 (103 MB)

SWAP OUT: P8 (135 MB)

SWAP IN : P1 (144 MB)

SWAP OUT: P7 (61 MB)

SWAP IN : P8 (135 MB)

SWAP OUT: P8 (135 MB)

SWAP IN : P9 (90 MB)

SWAP IN : P7 (61 MB)

SWAP OUT: P9 (90 MB)

SWAP OUT: P7 (61 MB)

SWAP IN : P8 (135 MB)

SWAP OUT: P8 (135 MB)

SWAP IN : P7 (61 MB)

SWAP OUT: P7 (61 MB)

SWAP IN : P8 (135 MB)

SWAP OUT: P8 (135 MB)

SWAP IN : P9 (90 MB)

SWAP OUT: P9 (90 MB)

SWAP IN : P8 (135 MB)

SWAP OUT: P8 (135 MB)

SWAP IN : P7 (61 MB)

SWAP OUT: P7 (61 MB)

SWAP IN : P8 (135 MB)

SWAP OUT: P8 (135 MB)

SWAP IN : P7 (61 MB)

=== SIMULATION END ===


```

=== SIMULATION START ===
SWAP IN : P1 (144 MB)
SWAP IN : P6 (110 MB)
SWAP IN : P8 (135 MB)
SWAP IN : P9 (90 MB)
SWAP IN : P2 (120 MB)
SWAP IN : P3 (158 MB)
SWAP IN : P7 (61 MB)
SWAP IN : P4 (103 MB)
SWAP OUT: P2 (120 MB)
SWAP IN : P5 (113 MB)
SWAP IN : P0 (109 MB)
SWAP OUT: P0 (109 MB)
SWAP OUT: P7 (61 MB)
SWAP IN : P2 (120 MB)
SWAP OUT: P2 (120 MB)
SWAP IN : P7 (61 MB)
SWAP IN : P0 (109 MB)
SWAP OUT: P0 (109 MB)
SWAP OUT: P7 (61 MB)
SWAP IN : P2 (120 MB)
SWAP OUT: P2 (120 MB)
SWAP IN : P7 (61 MB)
SWAP IN : P0 (109 MB)
SWAP OUT: P0 (109 MB)
SWAP OUT: P7 (61 MB)
SWAP IN : P2 (120 MB)
SWAP OUT: P2 (120 MB)
SWAP IN : P0 (109 MB)
SWAP IN : P7 (61 MB)
SWAP OUT: P0 (109 MB)
SWAP OUT: P7 (61 MB)
SWAP IN : P2 (120 MB)
SWAP OUT: P2 (120 MB)
SWAP IN : P0 (109 MB)
SWAP IN : P7 (61 MB)
SWAP OUT: P0 (109 MB)
SWAP OUT: P7 (61 MB)
SWAP IN : P2 (120 MB)
SWAP OUT: P2 (120 MB)
SWAP IN : P0 (109 MB)
SWAP IN : P7 (61 MB)
SWAP OUT: P0 (109 MB)
SWAP OUT: P7 (61 MB)
SWAP IN : P2 (120 MB)
SWAP OUT: P2 (120 MB)
SWAP IN : P0 (109 MB)
=== SIMULATION END ===

```

```

=== SUMMARY ===
P0 | Swap Count: 5
P1 | Swap Count: 0
P2 | Swap Count: 6
P3 | Swap Count: 0
P4 | Swap Count: 0
P5 | Swap Count: 0
P6 | Swap Count: 0
P7 | Swap Count: 5
P8 | Swap Count: 0
P9 | Swap Count: 0

```

PENJELASAN KODE

A. Struktur Process

```
typedef struct {  
    int pid;  
    int size;  
    int priority;  
    int in_memory;  
    int swap_count;  
    time_t last_access;  
    time_t load_time;  
} Process;
```

Fungsi tiap field:

- pid → ID proses
- size → ukuran memory (MB)
- priority → prioritas proses
- in_memory → status (0 = swap, 1 = RAM)
- swap_count → jumlah swap
- last_access → untuk **LRU**
- load_time → untuk **FIFO**

B. Mekanisme Swap

Swap Out

```
void swap_out(Process *p)
```

- Mengeluarkan proses dari RAM
- Mengurangi used_memory
- Menambah swap_count

Swap In

```
void swap_in(Process *p)
```

- Memasukkan proses ke RAM
- Update waktu akses dan load

C. Strategi Swap

1. FIFO

- Proses **paling lama berada di memory** dikeluarkan dulu

`select_victim_fifo()`

2. LRU

- Proses **paling lama tidak diakses** dikeluarkan

`select_victim_lru()`

3. Priority-based

- Proses dengan **prioritas terendah** dikeluarkan

`select_victim_priority()`

D. Simulasi

`simulate(strategy)`

- Akses proses secara random selama **100 time units**
- Jika memory tidak cukup → pilih victim → swap out
- Lalu swap in proses baru

3. CONTOH ANALISIS (UNTUK LAPORAN)

Tabel Performa (Contoh)

Strategy Swap Out Count Swap In Count Total Swap Time Avg Response Time

FIFO	Tinggi	Tinggi	Tinggi	Lambat
LRU	Rendah	Rendah	Rendah	Cepat
Priority	Sedang	Sedang	Sedang	Sedang

Jawaban Analisis

1. Strategi yang meminimalkan thrashing

- **LRU**, karena mempertahankan proses yang sering diakses.

2. Pengaruh ukuran memory

- Memory kecil → swap sering → thrashing
- Memory besar → swap jarang → performa naik

3. Kapan swapping lebih baik daripada killing process?

- Proses penting / jangka panjang
- Sistem multitasking
- Ingin mempertahankan state proses

TUGAS 6: ARCHITECTURE ANALYSIS

A. Intel Architecture Analysis

1. cek informasi cpu:

```
root@DESKTOP-J9C60BQ:~# lscpu | grep -E "Architecture|CPU(s)|Model name|Thread|Core"
Architecture:          x86_64
CPU(s):                8
On-line CPU(s) list:   0-7
Model name:            Intel(R) Core(TM) i7-10610U CPU @ 1.80GHz
Thread(s) per core:    2
Core(s) per socket:    4
NUMA node0 CPU(s):    0-7
root@DESKTOP-J9C60BQ:~# |
```

Analisis:

- Sistem menggunakan **arsitektur 64-bit (x86_64)**.
- Mendukung **multithreading (SMT / Hyper-Threading)**.
- Jumlah core fisik dan logical core menunjukkan dukungan paralelisme yang baik.

2. Cek page size:

```
root@DESKTOP-J9C60BQ:~# getconf PAGE_SIZE
4096
root@DESKTOP-J9C60BQ:~# |
```

Analisis:

- Ukuran page adalah **4 KB**.
- Ini adalah ukuran page standar pada Linux modern.
- Digunakan oleh OS untuk **virtual memory management** dan paging.

3. cek TLB info:

```
root@DESKTOP-J9C60BQ:~# cpuid | grep -i tlb
Command 'cpuid' not found, but can be installed with:
apt install cpuid
```

4. Cek interface chace:

```
root@DESKTOP-J9C60BQ:~# lscpu | grep -i cache
L1d cache:            128 KiB (4 instances)
L1i cache:            128 KiB (4 instances)
L2 cache:             1 MiB (4 instances)
L3 cache:             8 MiB (1 instance)
```

Analisis:

- **L1:** cache tercepat, dekat dengan core.
- **L2:** cache menengah.
- **L3:** shared antar core.
- Hirarki cache mengurangi latency akses memori utama.

B. memory information

1. total memory:

```
root@DESKTOP-J9C60BQ:~# free -h
```

	total	used	free	shared	buff/cache	available
Mem:	7.6Gi	452Mi	7.2Gi	3.4Mi	102Mi	7.2Gi
Swap:	2.0Gi	0B	2.0Gi			

2. virtual memory info

```
root@DESKTOP-J9C60BQ:~# vmstat 1 5
```

procs		-----memory-----				---swap--		-----io----		-system--				-----cpu-----			
r	b	swpd	free	buff	cache	si	so	bi	bo	in	cs	us	sy	id	wa	st	gu
0	0	0	7539724	1248	104224	0	0	536	48	160	0	0	0	100	0	0	0
0	0	0	7539724	1248	104412	0	0	0	0	39	49	0	0	100	0	0	0
0	0	0	7539724	1248	104412	0	0	0	0	23	29	0	0	100	0	0	0
0	0	0	7539724	1248	104412	0	0	0	0	22	34	0	0	100	0	0	0
0	0	0	7539724	1248	104412	0	0	0	0	51	49	0	0	100	0	0	0